

FedActuary: Architectural Blueprint & Codebase Guide

Comprehensive Micro-Level Educational Manual — From First Principles to Production Deployment

Target Audience: Software Engineers, Actuaries, Data Scientists, Students	Benchmark: French MTPL (freMTPL2freq, 678,013 policies)
Author / Project: FedActuary Research & Development Implementation	Foundation: ■mietanka, Liew, Hand, Loh, & Chen (BAJ, 2026)

1. Executive Overview: Why FedActuary Exists

In automobile insurance, premium pricing relies on estimating the expected claim frequency ($\lambda = \text{ClaimNb} / \text{Exposure}$). Actuaries require vast datasets to detect subtle risk interactions (e.g., driver age vs. vehicle power in dense metropolitan areas). However, European privacy regulations (GDPR), national anti-trust statutes, and commercial competition prohibit insurance companies from centralizing customer records into a single database. This creates **isolated data silos**, leaving smaller regional insurers with insufficient data, leading to severe under-pricing or over-pricing.

FedActuary solves this trilemma by pioneering **Horizontal Federated Learning (HFL)** with **Formal Differential Privacy (DP-SGD)** and **Post-Hoc Explainability (SHAP)**. Insurers collaborate by exchanging solely neural network parameter updates, guaranteeing that raw policyholder identities, geolocations, and driving records never leave private company servers.

Key Research & Engineering Objectives (PRD v1.0):

- **Reproduce Baseline:** Prove that collaborative Federated Learning matches centralized Global performance and substantially outperforms isolated Partial insurers.
- **Handle Real-World Heterogeneity:** Quantify performance collapse under non-IID Dirichlet label skew ($\alpha \in \{0.1, 0.5, 1, 5\}$) and regional feature skew ($\alpha = 0.5$), correcting divergence via client-side FedProx.
- **Formal Privacy Guarantees:** Calibrate Opacus DP-SGD across privacy budgets ($\epsilon \in \{\infty, 8, 3, 1\}$) and implement a novel frozen per-layer sensitivity clipping extension.
- **Actuarial Transparency:** Validate via SHAP GradientExplainer that risk factor attributions stay faithful across decentralized and privacy-constrained checkpoints (Spearman rank $r_s > 0.74$).

2. Micro-Level Glossary: Technical Terms Explained from Scratch

To ensure complete clarity for readers from software engineering, data science, and actuarial backgrounds, every core mathematical, actuarial, and algorithmic term is defined below at the micro level:

Term & Domain	Micro-Level Architectural Explanation
Exposure (v) [Actuarial]	The fraction of a year that an insurance policy was active (e.g., 0.5 = 6 months). Exposure is NOT an ordinary feature; claims scale linearly with time. A driver insured for a full year has twice the claim opportunity of an identical driver insured for 6 months.
ClaimNb (y) [Actuarial]	The discrete integer count of claims reported during the policy term {0, 1, 2, 3, 4}. In auto insurance, >95% of policies report exactly 0 claims, making the distribution highly zero-inflated and non-Gaussian.
Poisson Deviance (%PDE) [Actuarial Metric]	The primary measure of model fit for count data: d2_tweedie_score(power=1). Standard Mean Squared Error (MSE) assumes Gaussian errors and fails on counts. Deviance measures the likelihood ratio between the model and an intercept-only null. A positive %PDE indicates the model extracts genuine risk signals beyond the average frequency.
Lorenz Gini Coefficient [Actuarial Metric]	Measures risk sorting ability. Policies are ordered by predicted annual claim frequency ($y_hat / \text{Exposure}$). The Gini coefficient calculates the area between the cumulative exposure vs. cumulative claims curve (Lorenz curve). A higher Gini means the insurer correctly isolates high-risk drivers from safe drivers.

Bonus-Malus Index [Insurance Rating]	The European statutory merit-demerit rating system. Starts at 100. Each claim-free year reduces the score by 5% (down to 50, a 50% discount). Each fault accident increases the score by 25% (up to 150+). It serves as the single strongest historical proxy for driver risk.
Horizontal Federated Learning [FL Algorithm]	Collaborative machine learning where multiple institutions share the exact same feature schema (same 39 columns) but possess entirely distinct row populations (different drivers). Training occurs locally on client nodes, and only mathematical gradients/weights are shared.
FedAvg (Federated Averaging) [FL Algorithm]	The foundational algorithm introduced by McMahan et al. (2017). Each client trains locally for E epochs. The central server collects client weight matrices W_k and averages them proportional to client sample size: $W_{\text{global}} = \text{sum}((n_k / N) * W_k)$. Clients then synchronize to W_{global} .
Non-IID Heterogeneity [FL Challenge]	Non-Independent and Identically Distributed data. In practice, Insurer A might operate in Paris (high traffic density, small diesel cars) while Insurer B operates in rural Brittany (low density, agricultural usage). Local models naturally optimize in divergent gradient directions.
Dirichlet Distribution & Alpha [Probability]	A multivariate continuous distribution used to synthetically partition dataset rows among clients. The concentration parameter alpha (α) governs data skew. When alpha is low (0.1), a few clients monopolize almost all specific claim or region categories (extreme skew). When alpha is high (5.0), rows are distributed almost uniformly.
FedProx Regularization [FL Algorithm]	An extension of FedAvg for non-IID conditions (Li et al., 2020). Adds a proximal regularization term to each client's loss: $+ (\mu/2) * w - w_{\text{global}} ^2$. This penalizes local models from drifting too far away from the global consensus during local epochs. Crucially, FedProx runs client-side only without server changes.
Differential Privacy (DP) [Privacy Theory]	The gold-standard mathematical framework providing a provable bound on information leakage. An algorithm M is (ϵ, δ) -differentially private if the presence or absence of any single individual in the dataset changes the probability of any output by at most $e^\epsilon + \delta$.
Epsilon (ϵ) and Delta (δ) [Privacy Budget]	Epsilon (ϵ) is the privacy loss parameter. Lower ϵ means stronger privacy (less information leaked). Epsilon $= \infty$ represents standard non-private training; $\epsilon = 1.0$ represents high privacy. Delta ($\delta = 1e-5$) represents the tiny failure probability, which must be strictly smaller than $1 / N$.
DP-SGD & Opacus [DP Framework]	Differentially Private Stochastic Gradient Descent. In each mini-batch, Opacus clips each individual per-sample gradient to maximum norm C, sums them, and injects calibrated Gaussian noise: $N(0, \text{sigma}^2 * C^2 * I)$. This mathematically conceals outlier records.
Per-Layer Clipping Extension [Privacy Novelty]	Standard DP-SGD clips the flat L2-norm of the entire parameter vector. The per-layer extension computes layer-specific thresholds $C_l = C * (W_l / W_{\text{all}})$ frozen from a baseline checkpoint, ensuring $\text{sqrt}(\text{sum } C_l^2) = C$. This preserves critical output layer dynamics while maintaining the exact same total privacy budget.
SHAP & Shapley Values [Explainability]	A cooperative game theory technique developed by Lloyd Shapley (Nobel Prize) and adapted to machine learning by Lundberg & Lee. It calculates the fair marginal contribution of each input feature to a specific prediction. The sum of all SHAP values equals the difference between the model output and the expected baseline.
Spearman Rank Correlation (r_s) [Statistics]	A non-parametric metric evaluating the monotonicity of rankings between two models. Used in Phase 4 to verify whether Federated, Non-IID, and Privacy-constrained models rank the importance of insurance risk factors in the same order as the centralized Global model.

3. End-to-End Sequential Pipeline & Execution Lifecycle

The FedActuary system is strictly organized into a 10-step deterministic sequence. Understanding this order reveals how raw data flows from disk into sanitized tensors, distributed shards, federated optimization loops, privacy accountants, explainability engines, and finally the interactive web UI:

Step	Phase / Role	Primary File	Micro-Level Execution Logic
Step 1	Raw Ingestion & Deterministic Sanitization	src/data.py	Loads freMTPL2freq.csv (678,013 rows). Caps ClaimNb at 4, VehAge at 20, DrivAge at 90, BonusMalus at 150, Exposure at 1.0. Applies natural log to Density. Encodes Area to integers 1-6. Encodes VehGas to 1 (Regular) / 2 (Diesel). Fixes canonical categories for VehBrand (11 levels) and Region (22 levels). Expands dummy variables (drop_first=True) to produce exactly 39 feature columns.

Step 2	Deterministic Stratification & Scaler Fitting	src/data.py	Splits the 678,013 rows into Train (~72%), Validation (~8%), and Test (~20%) using seed 42. Fits MinMaxScaler ONLY on the training split numerical columns [Area, VehPower, VehAge, DrivAge, BonusMalus, Density]. Transforms Val and Test without leaking information. Persists scaler.pkl and feature_columns.json.
Step 3	Neural Architecture Definition	src/model.py	Defines MultipleRegression: Input(39) -> Linear(39, 15) -> Tanh -> Linear(15, 5) -> Tanh -> Linear(5, 1) -> exp(). Exactly 686 trainable parameters. All linear layer weights are initialized via Xavier Uniform; biases are initialized to zeros. The exponential output link guarantees strictly positive predicted claim counts.
Step 4	Training Core & Optimizer Moments	src/train.py & src/metrics.py	Instantiates Client objects owning private NAdam optimizers (lr=0.001) and PoissonNLLLoss(log_input=False, full=True). Computes Tweedie deviance (%PDE) and Lorenz Gini ordering. Gradient clipping is locked at max L2-norm = 5.0.
Step 5	Phase 1: Baseline Reproduction	phase1_baseline.py	Executes the three fundamental benchmarks: Global (pooled training), Partial (10 isolated models with equal local epoch budget), and Federated (FedAvg over 10 insurers for 350 rounds x 10 local epochs). Persists global.pt, partial_0..9.pt, federated.pt, and phase1_metrics.json.
Step 6	Phase 2: Non-IID Dirichlet & FedProx Sweeps	phase2_nonIID.py	Partitions training data using Dirichlet distributions. Tests label-shift across ClaimNb (alpha in {0.1, 0.5, 1, 5}) and feature-shift across Region (alpha = 0.5). Evaluates standard FedAvg vs. client-side FedProx (mu = 0.01). Persists non_iid_alpha_0.5.pt and phase2_heterogeneity.json.
Step 7	Phase 3: Formal Differential Privacy via Opacus	phase3_privacy.py	Attaches Opacus PrivacyEngine to local training loops. Computes calibrated Gaussian noise multipliers for target epsilons {inf, 8, 3, 1} with delta = 1e-5. Evaluates per-layer clipping extension at epsilon = 3.0. Persists dp_eps_*.pt and phase3_privacy_utility.json.
Step 8	Phase 4: Post-Hoc Explainability via SHAP	phase4_explainability.py	Constructs shap.GradientExplainer with reference background ceiling <= 50. Explains predictions across 5 key models (Global, Federated, Non-IID, DP-3.0, DP-1.0). Computes pairwise Spearman rank correlations. Persists shap_background.npy and feature_rank.json.
Step 9	One-Command Orchestration & Automatic Pause/Resume	run_pipeline.py	Master runner that executes Phase 1 through Phase 4 in sequence. Automatically serializes checkpoint state every 10 rounds/epochs. If interrupted (power cut, terminal closed, manual pause), rerunning resumes smoothly from the exact round without loss.
Step 10	Interactive Dashboard & Inference Tool	app.py	Streamlit application offering a Welcome Hero, real-time 12-field policy prediction with live SHAP attribution, unified Master Specifications Table, Feature Glossary with category tables, Pareto curves, and in-browser live FL simulation.

4. Comprehensive File-by-File Blueprint

Every file in the repository serves a specific mathematical or architectural function. Below is an exhaustive breakdown of every module, script, checkpoint, and data asset:

File: src/__init__.py	Role: Package Initializer
<i>Consumes: None</i>	<i>Generates: Python package symbol export</i>
Architectural Mechanics: Marks the src/ folder as an importable Python package, enabling clean absolute and relative imports across scripts (e.g., from src.data import load_raw_features). Contains zero execution overhead.	
File: src/data.py	Role: Data Ingestion, Cleaning & Preprocessing
<i>Consumes: data/freMTPL2freq.csv</i>	<i>Generates: scaler.pkl, feature_columns.json, train/val/test splits</i>

Architectural Mechanics: The foundation of the entire pipeline. Implements `load_raw_features()` which enforces domain capping (`ClaimNb<=4`, `VehAge<=20`, `DrivAge<=90`, `BonusMalus<=150`, `Exposure<=1.0`), log transforms `Density`, maps `Area` and `VehGas`, and extracts one-hot dummy variables across canonical `VEH_BRANDS` (11 levels) and `REGIONS` (22 levels). Implements `split_and_scale()` which fits `MinMaxScaler` exclusively on train rows and returns `split_indices` for exact row tracking. Implements `preprocess_single_record()` for single-row live inference using the persisted scaler without refitting.

File: `src/model.py`**Role: Neural Network Architecture Definition***Consumes: Feature count (39)**Generates: MultipleRegression PyTorch nn.Module*

Architectural Mechanics: Defines the exact feedforward architecture from ■mietanka et al. (2026): `Input(39) -> Linear(39, 15) -> Tanh -> Linear(15, 5) -> Tanh -> Linear(5, 1) -> torch.exp()`. Contains exactly 686 trainable parameters. All linear weights are initialized with Xavier Uniform; biases initialized to zeros. The exponential activation on the single output unit guarantees non-negative predicted Poisson claim counts.

File: `src/train.py`**Role: Training Engine, Federated Averaging & Pause/Resume Core***Consumes: Tensors, MultipleRegression, state dicts**Generates: Client class, `fedavg()`, `run_federated()`, `safe_load_torch()`*

Architectural Mechanics: Encapsulates client-side training and server-side aggregation. Client maintains local data tensors, private `MultipleRegression` model, persistent `NAdam` optimizer, and `PoissonNLLLoss`. Supports FedProx via proximal loss term $+\frac{(\mu/2)*||w - w_{global}||^2}$. Includes gradient norm clipping ($\max L2 = 5.0$). Implements sample-weighted `fedavg()`. `run_federated()` manages the round loop with seamless pause & resume checkpointing and `safe_load_torch()` for PyTorch 2.6+ unpickling compatibility.

File: `src/metrics.py`**Role: Actuarial Evaluation Metrics***Consumes: `y_true`, `y_pred`, `exposure`**Generates: %PDE (`percentage_deviance_explained`), Lorenz Gini*

Architectural Mechanics: Implements the true actuarial evaluation criteria. `percentage_deviance_explained()` uses scikit-learn's `d2_tweedie_score(power=1, sample_weight=exposure) * 100` to evaluate Poisson deviance reduction compared directly to raw `ClaimNb`. `gini_coefficient()` orders policies by implied annual claim frequency (`y_pred / exposure`) and computes the Lorenz curve integral to evaluate risk ranking ability independent of policy duration.

File: `src/partition.py`**Role: Non-IID Dirichlet Data Partitioner***Consumes: Labels / features array, client count, alpha**Generates: List of client index arrays*

Architectural Mechanics: Implements `dirichlet_partition_by_label()` which samples class proportions from `Dirichlet(alpha, ..., alpha)` for each distinct label or feature category. Distributes rows among clients with starvation protection (`min_size=10` per insurer). Supports both label-shift (`ClaimNb`) and feature-shift (`Region`).

File: `phase1_baseline.py`**Role: Phase 1: Baseline Reproduction Runner***Consumes: `data/freMTPL2freq.csv`**Generates: `checkpoints/global.pt`, `partial_0..9.pt`, `federated.pt`, `results/phase1_metrics.json`*

Architectural Mechanics: Runs the primary benchmark comparisons: Centralized Global baseline (350 epochs), 10 isolated Partial insurers (350 rounds x 10 epochs = 3,500 total local epochs), and 10-insurer Federated FedAvg (350 rounds x 10 local epochs). Features epoch-by-epoch and round-by-round resume support via `--resume`.

File: `phase2_nonIID.py`**Role: Phase 2: Data Heterogeneity & FedProx Sweeps***Consumes: `data/freMTPL2freq.csv`, `train split indices`**Generates: `checkpoints/non_iid_alpha_0.5.pt`, `results/phase2_heterogeneity.json`*

Architectural Mechanics: Sweeps non-IID conditions: `ClaimNb` label-shift across `alpha` in {0.1, 0.5, 1.0, 5.0} and `Region` feature-shift across `alpha = 0.5`, testing both FedAvg ($\mu=0.0$) and client-side FedProx ($\mu=0.01$). Automatically resumes incomplete sweeps upon restart.

File: `phase3_privacy.py`**Role: Phase 3: Opacus DP-SGD & Per-Layer Sensitivity***Consumes: `data/freMTPL2freq.csv`, `checkpoints/global.pt`**Generates: `checkpoints/dp_eps_*.pt`, `results/phase3_privacy_utility.json`*

Architectural Mechanics: Applies differential privacy via Opacus DP-SGD. Calibrates Gaussian noise multipliers for target epsilons {inf, 8.0, 3.0, 1.0} with delta = 1e-5. Implements the frozen per-layer sensitivity clipping extension at epsilon = 3.0, allocating layer thresholds CI proportional to Phase 1 weight norms while maintaining the exact same total privacy budget.	
File: phase4_explainability.py <i>Consumes:</i> checkpoints/*.pt, freMTPL2freq test set	Role: Phase 4: SHAP Feature Attribution & Correlation <i>Generates:</i> checkpoints/shap_background.npy, results/feature_rank.json
Architectural Mechanics: Executes shap.GradientExplainer with reference background ceiling bounded strictly to <= 50. Computes mean absolute feature attributions across 5 models (Global, Federated, Non-IID, DP-3.0, DP-1.0) and calculates pairwise Spearman rank correlation matrix.	
File: run_pipeline.py <i>Consumes:</i> All phase scripts	Role: Master End-to-End Pipeline Orchestrator <i>Generates:</i> Full production checkpoints and results
Architectural Mechanics: A single command (python run_pipeline.py) that executes Phase 1 through Phase 4 on all 678,013 policies in deterministic order. Forwards --resume flags, tracks subprocess exit codes, and provides accurate stage-by-stage runtime telemetry.	
File: app.py <i>Consumes:</i> checkpoints/, results/, freMTPL2freq.csv	Role: Interactive Streamlit Web Dashboard <i>Generates:</i> Interactive browser dashboard on port 8501
Architectural Mechanics: Production web interface structured into 8 modular pages: Welcome & Overview Hero, Interactive 12-field Policy Prediction Tool with live SHAP attribution bar charts, unified Master Specifications Table, Policy Feature Guide & Glossary with categorical tables, Data Heterogeneity curves, Differential Privacy Pareto frontiers, Explainability rank matrices, and in-browser Live FL simulation.	
File: requirements.txt <i>Consumes:</i> pip package manager	Role: Dependency Specification <i>Generates:</i> Installed Python environment
Architectural Mechanics: Declares compatible version ranges for PyTorch, Flower, scikit-learn, Pandas, NumPy, SciPy, Opacus, SHAP, and Streamlit. Designed for 100% portability across Windows and Streamlit Community Cloud (Linux).	
File: .gitignore <i>Consumes:</i> Git repository	Role: Git Version Control Configuration <i>Generates:</i> Clean tracking index
Architectural Mechanics: Excludes temporary files (__pycache__, virtual environments, IDE files) while explicitly ensuring that data/freMTPL2freq.csv (31.5MB), checkpoints/, and results/ remain fully tracked and committable.	
File: README.md <i>Consumes:</i> Markdown viewer / GitHub	Role: Project Overview & Quickstart Guide <i>Generates:</i> Repository homepage documentation
Architectural Mechanics: Provides project badges, architecture diagrams, benchmark tables, installation steps, reproduction commands, pause/resume instructions, and GitHub to Streamlit Cloud deployment guides.	
File: REPORT.md <i>Consumes:</i> Markdown viewer / Academic reader	Role: Comprehensive Academic Research Report <i>Generates:</i> Research documentation & paper analysis
Architectural Mechanics: Complete formal research paper detailing methodology, Poisson deviance derivation, non-IID Dirichlet formulation, DP-SGD accounting, empirical findings, and academic citations [1]-[5].	

5. Checkpoints & Results Directory Artifacts

All learned parameters and evaluation metrics are permanently stored on disk to enable offline dashboard operation without retraining:

Artifact Path	Format & Size	Permanent Role & Function
checkpoints/global.pt	PyTorch State Dict (5.6 KB)	Weights of the centralized model trained on all pooled training data for 350 epochs. Serves as the gold-standard upper bound for predictive deviance.

checkpoints/federated.pt	PyTorch State Dict (5.7 KB)	Weights of the 10-insurer collaborative model trained via FedAvg for 350 rounds x 10 local epochs. Matches global utility within ~0.23% deviance.
checkpoints/partial_0.pt .. partial_9.pt	10 PyTorch State Dicts (5.7 KB each)	Weights of 10 isolated insurer models trained exclusively on their private 10% data shards without collaboration. Illustrates the penalty of data silos.
checkpoints/non_iid_alpha_0.5.pt	PyTorch State Dict (5.8 KB)	Weights of the collaborative model trained under severe Dirichlet ClaimNb label skew ($\alpha = 0.5$), demonstrating model robustness under heterogeneity.
checkpoints/dp_eps_*.pt	4 PyTorch State Dicts (5.7 KB each)	Models trained with Opacus DP-SGD across privacy budgets $\epsilon \in \{\infty, 8.0, 3.0, 1.0\}$ with $\delta = 1e-5$. Demonstrates the privacy-utility trade-off.
checkpoints/dp_per_layer_eps_3.0.pt	PyTorch State Dict (5.9 KB)	Model trained using the per-layer sensitivity clipping extension at $\epsilon = 3.0$. Shows superior deviance explained compared to flat gradient clipping.
checkpoints/scaler.pkl	Scikit-Learn MinMaxScaler (0.9 KB)	Fitted MinMaxScaler parameters (min and scale vectors) for the 6 continuous columns. Used to normalize test records and live user queries.
checkpoints/feature_columns.json	JSON Schema (1.0 KB)	The canonical list of all 39 feature names in strict columnar order. Guarantees input alignment between training tensors and single-record inference.
checkpoints/shap_background.npy	NumPy Array (7.9 KB)	The 50 representative background policies sampled from the test split. Used by shap.GradientExplainer to compute live marginal risk attributions.
results/phase1_metrics.json & .csv	JSON & CSV (1.7 KB)	Numerical %PDE and Gini benchmarks for Global, Partial, and Federated models, plus round-by-round convergence trajectories.
results/phase2_heterogeneity.json	JSON Results (8.0 KB)	Complete metric records for all 10 Dirichlet sweeps across ClaimNb and Region under both FedAvg and FedProx.
results/phase3_privacy_utility.json	JSON Results (1.3 KB)	Empirical Pareto frontier data comparing flat DP-SGD vs. per-layer DP clipping across all privacy budget levels.
results/feature_rank.json	JSON Results (15.4 KB)	Mean absolute SHAP attributions for all 39 features across 5 checkpoints, plus the complete pairwise Spearman rank correlation matrix.

6. The Checkpointing Pause & Resume Engine

A critical feature engineered into FedActuary is **fault-tolerant resumption**. Training on 678,013 policies across 350 rounds requires ~3 hours on a multi-core CPU. The system guarantees that training can be interrupted at any moment and resumed seamlessly without losing progress:

How Fault-Tolerant Resumption Operates Mathematically:

- Periodic State Serialization:** Every 10 rounds/epochs, `run_federated()` serializes a complete state tuple: `{round, global_state, client_states, history}` to `checkpoints/p1_federated_resume.pt`.
- Momentum Continuity:** Crucially, `client_states` captures not just model weights, but each client's internal **NAdam optimizer state_dict** (first moment m_t , second moment v_t , and step count). This ensures gradient momentums do not reset to zero upon restart.
- Seamless Restoration:** When restarted, the script detects the checkpoint, restores client optimizer moments, initializes round counter to `saved_round + 1`, and continues. Resuming from round 140 to 141 is **mathematically identical** to running continuously.
- Automatic Cleanup:** Once all 350 rounds finish, the temporary resume checkpoint is cleanly deleted, leaving only the permanent production weights.

7. Streamlit Web Dashboard (app.py) Architecture

The user-facing dashboard is designed for underwriters, actuaries, and executive stakeholders. It provides immediate offline responsiveness by consuming precomputed checkpoints and results:

Dashboard Navigation Tab	Functionality & Actuarial Value
■ Welcome & Overview	Action hub welcoming users with quick-launch cards to query policies, inspect model specs, or explore privacy curves.

■ Interactive Policy Prediction	Form with 12 fields (BonusMalus, VehAge, DrivAge, Exposure, Area, VehGas, VehBrand, Region, etc.). Computes expected claim counts and triggers live SHAP GradientExplainer attribution bar charts.
■ Model Details & Specifications	Master reference table detailing all 18 hyperparameter specifications, Poisson loss formulation, Xavier initialization, and published benchmark comparisons.
■ Policy Feature Guide & Glossary	Comprehensive educational dictionary explaining every input field, statutory rating bands, vehicle brand groupings, and French administrative region codes.
■ Data Heterogeneity (Non-IID)	Interactive Dirichlet distribution visualizers showing client sample imbalance and Pareto comparisons between FedAvg and client-side FedProx.
■ Differential Privacy (DP-SGD)	Privacy-utility trade-off curves across epsilon targets, highlighting the +0.87% deviance gain from the frozen per-layer sensitivity clipping extension.
■ Explainability (SHAP)	Feature importance bar charts and pairwise Spearman rank correlation heatmap comparing risk factor consistency across 5 checkpoints.
■ Live FL Simulation	In-browser live simulation demonstrating real-time collaborative training across 3 simulated insurers over 10 rounds.

8. Quick Reference Operations Manual

Desired Operation	Terminal Command (from fedactuary/)
Launch Web Application	<code>python -m streamlit run app.py</code>
Run Full Production Pipeline (with Pause/Resume)	<code>python run_pipeline.py</code>
Force Full Pipeline from Scratch (No Resume)	<code>python run_pipeline.py --no-resume</code>
Run Phase 1 Baseline Only	<code>python phase1_baseline.py</code>
Run Phase 2 Non-IID Sweeps Only	<code>python phase2_nonIID.py</code>
Run Phase 3 Differential Privacy Only	<code>python phase3_privacy.py</code>
Run Phase 4 SHAP Explainability Only	<code>python phase4_explainability.py</code>
Deploy to Streamlit Cloud	Set Main File Path to: app.py on share.streamlit.io